AWS Cloud Security Lab Report

Lab 1: S3 Bucket Misconfiguration — Public Exposure, Detection & Remediation

Prepared by: Om Fulsundar (may 2026)

Date	May 2026
Region	ap-south-1 (Mumbai)
Bucket Name	my-company-sensitive-data-om
Lab Type	S3 Security / Data Exposure
Severity	CRITICAL — CVSS 9.1

1. Lab Overview

This lab simulates one of the most impactful and unfortunately common cloud misconfigurations — a publicly exposed S3 bucket containing sensitive data. I created an S3 bucket called my-company-sensitive-data-om, intentionally disabled Block Public Access, and attached a bucket policy that made all objects readable by anyone on the internet. I then uploaded fake sensitive files to confirm the exposure, used AWS Config to detect the compliance violation, and finally remediated the issue by re-enabling Block Public Access and removing the permissive bucket policy.

The reason this lab matters is straightforward: S3 buckets are one of the most frequently misconfigured resources in AWS, and the consequences of a public bucket containing sensitive data can be devastating — regulatory fines, customer data exposure, reputational damage, and legal liability. This is not a theoretical exercise. High-profile companies have made exactly this mistake, and the damage was real.

> *Real-world relevance: The Capital One breach (2019), GoDaddy data exposure, and Twitch source code leak all involved misconfigured cloud storage permissions. Billions of records have been exposed globally through public S3 buckets.*

2. Objectives

Going into this lab, here's what I set out to do:

- Create an S3 bucket with Block Public Access intentionally disabled to simulate a misconfigured storage resource.
- Attach a public bucket policy (s3:GetObject for Principal: *) to make all objects accessible without authentication.
- Upload fake sensitive files (employee data and API keys) and verify they are publicly readable from an incognito browser.
- Use AWS Config with the s3-bucket-public-read-prohibited managed rule to detect and flag the misconfiguration as NONCOMPLIANT.
- Remediate the exposure by enabling Block All Public Access and removing the permissive bucket policy.
- Validate the fix by re-running the AWS Config evaluation (COMPLIANT) and confirming AccessDenied when accessing the file from the incognito browser.

3. Tools and Services Used

Tool / Service	Purpose in This Lab
Amazon S3 (Simple Storage Service)	Core storage service where the misconfigured bucket was created and files were uploaded.
S3 Bucket Policy	JSON-based access control mechanism used to grant public read access — the root misconfiguration.
Block Public Access	S3 security feature that overrides bucket policies to prevent any public access; disabled initially, then re-enabled for remediation.
AWS Config	Compliance monitoring service used to detect the NONCOMPLIANT bucket via the s3-bucket-public-read-prohibited managed rule.
Incognito Browser	Used to simulate unauthenticated public access to the bucket files, confirming exposure and then verifying the fix.

4. Lab Environment Setup

The lab ran entirely on a live AWS account in the ap-south-1 (Mumbai) region. No sandbox or emulated environment was used — this was real AWS infrastructure. I accessed the S3 console and AWS Config console through the browser, and used an incognito window on the same machine to simulate unauthenticated (public) access to the bucket objects.

For the sensitive data files, I created two simple text files locally before uploading them to the bucket. The content was fake but realistic enough to represent the kind of data that real organizations accidentally expose:

- employee-data.txt — contained a sample employee record: EMP001, Om Fulsundar, Salary: 50000
- api-keys.txt — contained a fake API key string: API_KEY=fake123456789

AWS Config was already enabled in the account. I just needed to add the specific rule for S3 public read detection and run the evaluation after the bucket was in a misconfigured state.

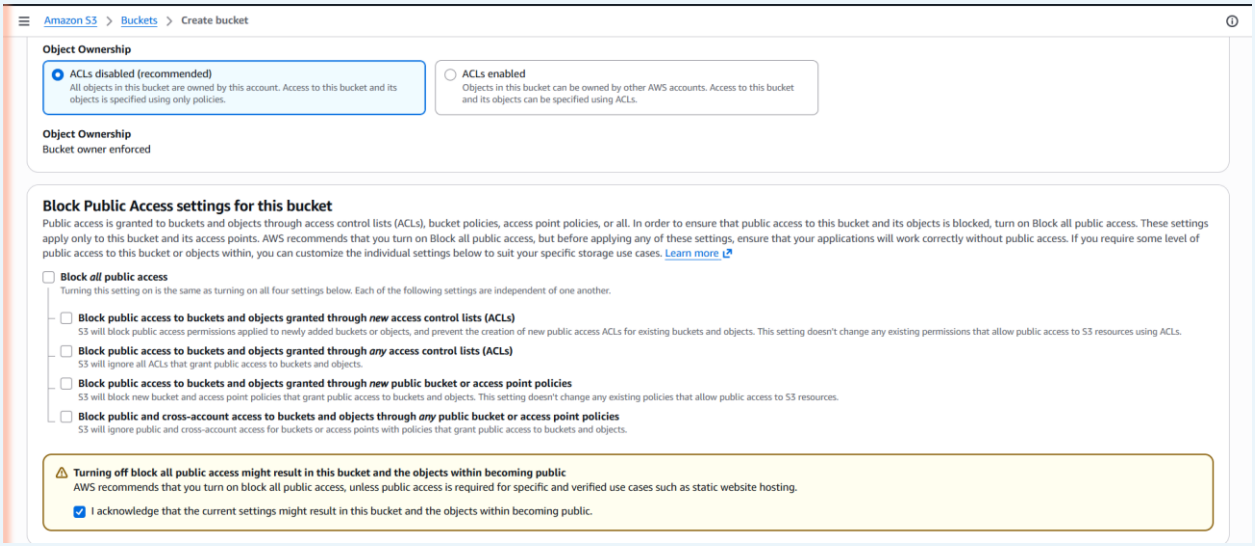
5. Step-by-Step Walkthrough

Phase 1 — Bucket Creation with Block Public Access Disabled

I navigated to S3 > Create bucket and set the bucket name to my-company-sensitive-data-om in the ap-south-1 region. The critical misconfiguration happened in the Block Public Access settings section. By default, AWS checks all four Block Public Access options — which is the right behavior. I unchecked all of them, which opened the bucket up to potential public exposure. AWS showed a warning: 'Turning off block all public access might result in this bucket and the objects within becoming public.' I had to check an acknowledgment box to proceed.

That acknowledgment checkbox is actually a meaningful friction point — it means that making a bucket public requires a conscious decision. In practice, though, developers under deadline pressure or without proper security training often just click through it without understanding the implications.

[SS1 — Bucket Creation with Block Public Access Unchecked]



S3 Create Bucket console showing all Block Public Access options unchecked and acknowledgment checkbox ticked

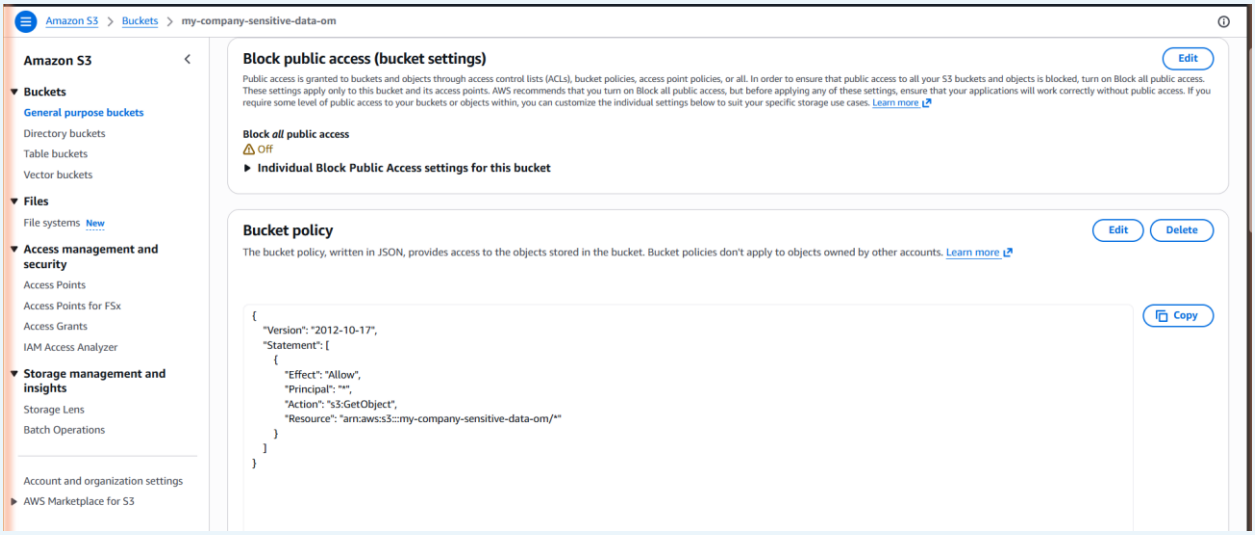
Phase 2 — Adding the Public Bucket Policy

After the bucket was created, I went to Permissions > Bucket policy and added the following JSON policy. This is the policy that actually makes objects publicly accessible to anyone on the internet without any authentication:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-company-sensitive-data-om/*"
    }
  ]
}
```

The Principal: "*" is the dangerous part here. It means literally any entity — any person, any bot, any automated scanner — can call s3:GetObject on any object in the bucket. There is no IP restriction, no authentication requirement, no rate limiting. Combined with the disabled Block Public Access, this creates a fully open bucket.

[SS2 — Bucket Policy Added (Public Access Policy)]



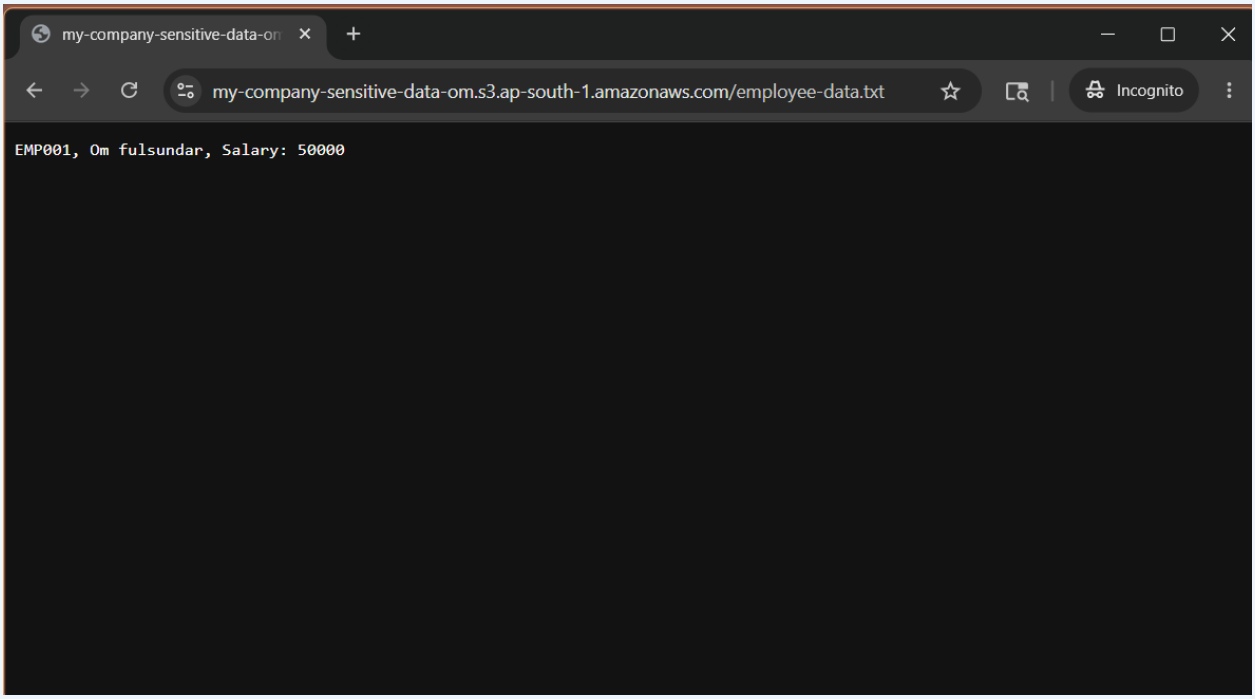
S3 Permissions tab showing Block Public Access OFF and the public s3:GetObject bucket policy applied to my-company-sensitive-data-om

Phase 3 — Verifying the Exposure

With the bucket open and the policy applied, I uploaded employee-data.txt and then opened an incognito browser window. I navigated directly to the public S3 URL: https://my-company-sensitive-data-om.s3.ap-south-1.amazonaws.com/employee-data.txt

The file loaded immediately with no login prompt, no token, nothing. The browser displayed the raw text content: EMP001, Om Fulsundar, Salary: 50000. This confirmed the misconfiguration was fully live and the data was accessible to anyone who knew — or could guess — the URL.

[SS3 — Incognito Browser Showing File Publicly Accessible]



Incognito browser window accessing employee-data.txt via the S3 public URL, displaying EMP001, Om Fulsundar, Salary: 50000

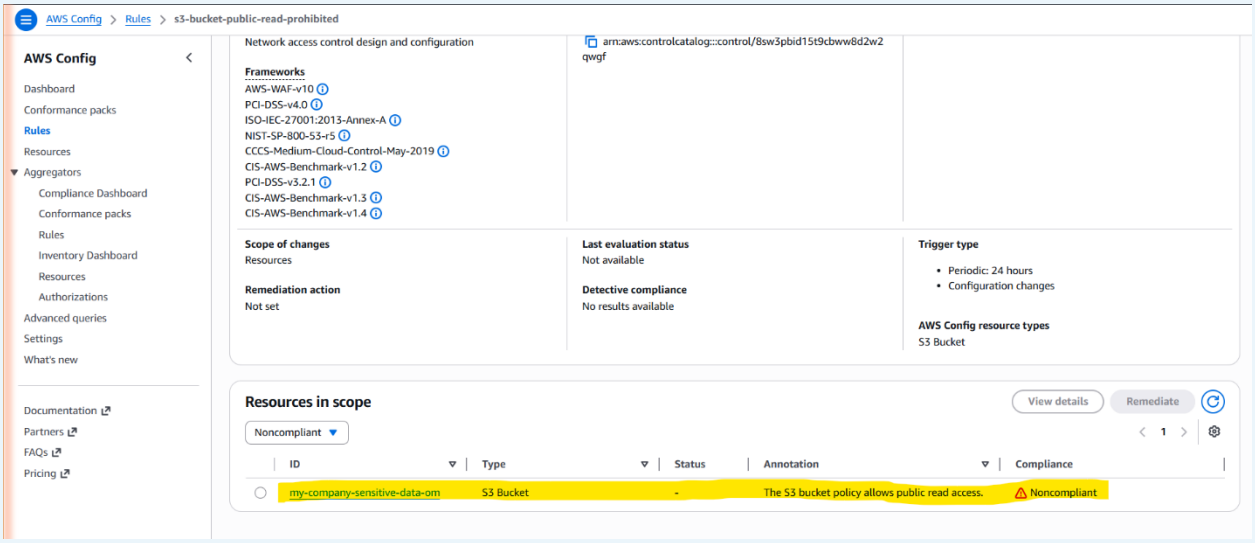
> *This is precisely how real data breaches happen. S3 bucket URLs follow a predictable format, and automated scanning tools continuously probe the internet for open buckets. A misconfigured bucket can be found and accessed within minutes of creation.*

Phase 4 — Detection Using AWS Config

I opened AWS Config and navigated to Rules, then added the AWS-managed rule s3-bucket-public-read-prohibited. This rule evaluates whether S3 buckets are configured to allow public read access, and it's mapped to multiple security frameworks including AWS-WAF-v10, PCI-DSS-v4.0, ISO-IEC-27001-2013, NIST-SP-800-53-r5, CCCS-Medium, and multiple CIS-AWS Benchmarks.

I triggered a manual evaluation. The result came back quickly: my-company-sensitive-data-om was flagged as NONCOMPLIANT with the annotation 'The S3 bucket policy allows public read access.' The bucket showed up in the Resources in Scope panel highlighted in yellow under the Noncompliant filter. This is exactly the kind of finding a cloud security team would investigate.

[SS4 — AWS Config Rule: NONCOMPLIANT Finding]



AWS Config s3-bucket-public-read-prohibited rule showing my-company-sensitive-data-om as NONCOMPLIANT with annotation: The S3 bucket policy allows public read access

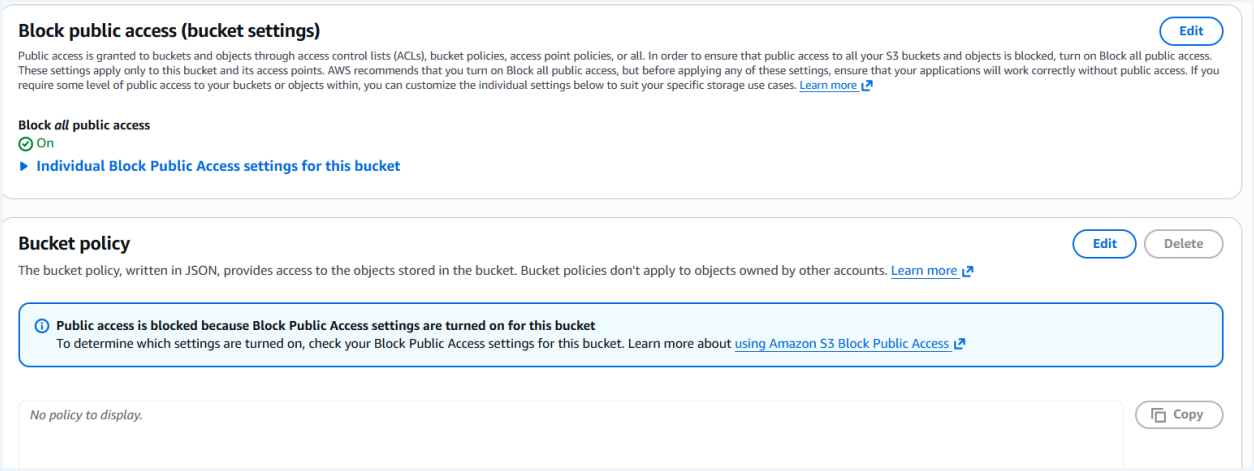
One thing I found interesting here: the Config rule is tagged against frameworks like PCI-DSS and NIST. That connection between a specific misconfiguration and a compliance framework is useful in real SOC or GRC work — it tells you not just that something is wrong, but why it matters from a regulatory standpoint.

Phase 5 — Remediation

Remediation was a two-step process. First, I went to S3 > my-company-sensitive-data-om > Permissions and enabled Block All Public Access. The toggle switched from Off to On, and the console immediately showed a blue notice: 'Public access is blocked because Block Public Access settings are turned on for this bucket.' The bucket policy was still technically there, but with Block Public Access enabled, S3 was now overriding it.

As a second step, I also deleted the bucket policy entirely. Leaving a permissive policy in place — even one being overridden by Block Public Access — is a risk. If someone later accidentally turns off Block Public Access again, the policy would immediately re-expose everything. Cleaning it up is the right thing to do.

[SS6 — Remediation: Block Public Access Enabled]

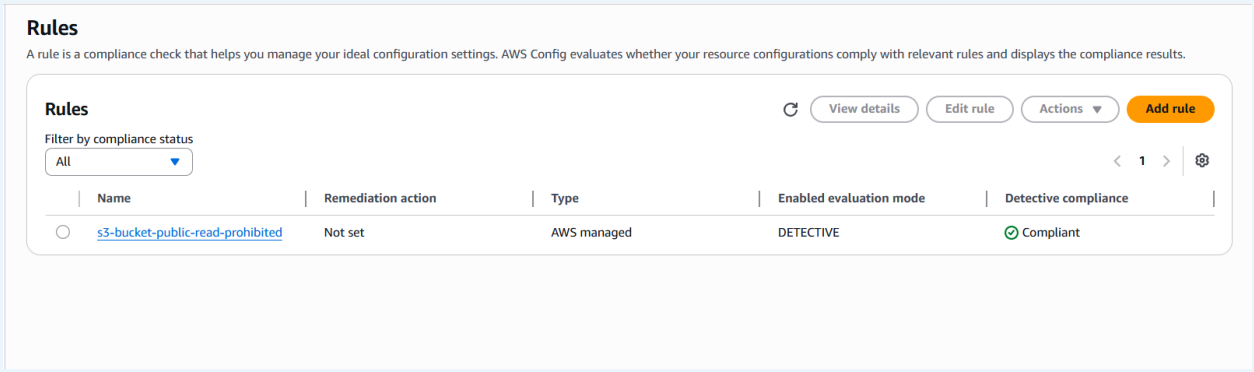


S3 Permissions tab showing Block All Public Access turned ON and bucket policy cleared with 'No policy to display' message

Phase 6 — Re-Evaluation and Validation

After remediation, I went back to AWS Config and triggered a re-evaluation of the s3-bucket-public-read-prohibited rule. This time the result was COMPLIANT — the green checkmark appeared next to the rule name and the bucket was no longer in the noncompliant resources list.

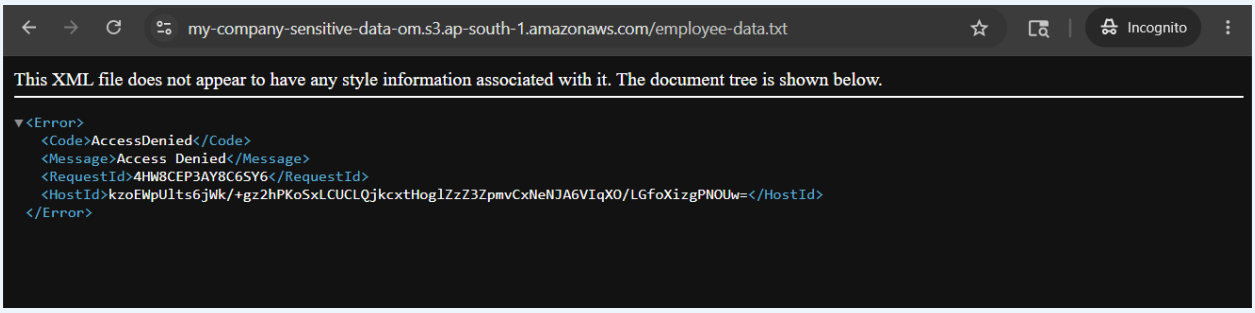
[SS7 — AWS Config After Remediation: COMPLIANT]



AWS Config Rules list showing s3-bucket-public-read-prohibited with green Compliant status after remediation

I also re-tested access from the incognito browser. Hitting the same URL — https://my-company-sensitive-data-om.s3.ap-south-1.amazonaws.com/employee-data.txt — now returned an XML error response with AccessDenied. The file was completely inaccessible to unauthenticated requests.

[SS8 — Incognito Browser After Fix: Access Denied]



Incognito browser showing XML AccessDenied error when accessing employee-data.txt after Block Public Access was re-enabled

6. Findings

Finding	S3 bucket my-company-sensitive-data-om was publicly readable by anyone on the internet with no authentication required.
Vulnerability Type	S3 Public Bucket Misconfiguration / Broken Access Control
Severity	CRITICAL
CVSS Score	9.1 — High (Network-accessible, no auth required, high confidentiality impact)
Affected Bucket	arn:aws:s3:::my-company-sensitive-data-om
Exposed Files	employee-data.txt, api-keys.txt
Region	ap-south-1 (Mumbai)
Root Cause	Block Public Access disabled + Principal: * bucket policy granting s3:GetObject
Config Rule Triggered	s3-bucket-public-read-prohibited (NONCOMPLIANT)

Real-World Impact

If this bucket contained real data and an attacker discovered it, the consequences could include:

- Direct access to employee PII (names, salaries, IDs) leading to GDPR/PDPA regulatory violations and potential fines.
- Exposure of API keys enabling attackers to authenticate as legitimate services or users — potentially leading to further account compromise.
- Automated S3 scanning bots actively crawl the internet for open buckets; discovery could happen within minutes of the misconfiguration.

- Competitor intelligence gathering or targeted social engineering using the exposed employee data.
- Reputational damage and customer trust erosion if the exposure becomes public.

> *Real precedents: The Capital One breach involved 100 million customer records exposed via a misconfigured cloud resource. GoDaddy accidentally exposed 1.2 million customer accounts through a similar S3 misconfiguration. These are not edge cases — they are predictable outcomes of skipping access control hygiene.*

7. Detection Method

Detection was achieved using AWS Config with the s3-bucket-public-read-prohibited managed rule. Here's how the detection chain worked:

1. AWS Config continuously evaluates S3 bucket configurations against the rule, triggered both periodically (every 24 hours) and on configuration changes — so any new bucket policy or permission change would trigger an evaluation automatically.
2. The rule checked both the Block Public Access setting (which was Off) and the bucket policy (which granted s3:GetObject to Principal: *). Either condition alone would have been sufficient to flag the bucket.
3. The evaluation returned a NONCOMPLIANT finding with a clear annotation: 'The S3 bucket policy allows public read access.' This annotation is directly actionable — it tells you exactly which control failed.
4. The Config finding is mapped to major compliance frameworks (PCI-DSS, NIST, CIS), which means this single finding could represent multiple control failures in a formal audit context.

In a production environment, this Config finding would ideally feed into a SIEM or ticketing system automatically. Teams can also configure AWS Config to trigger SNS notifications or Lambda remediations when a noncompliant finding is created — enabling near-real-time auto-remediation for known-bad configurations like this one.

> *AWS Macie is another service worth mentioning here. It uses machine learning to discover and classify sensitive data in S3 buckets, and would have flagged the employee-data.txt and api-keys.txt files as containing PII and credentials respectively — adding another detection layer on top of Config.*

8. Remediation

Step 1 — Enable Block All Public Access

In S3 > Permissions > Block public access, I toggled Block all public access to On. This is a bucket-level override that prevents any bucket policy or ACL from granting public access,

regardless of what those policies say. It's the single most effective control for preventing accidental public exposure.

Step 2 — Remove the Public Bucket Policy

I deleted the permissive bucket policy entirely. Relying solely on Block Public Access without cleaning up the policy would create a false sense of security — if Block Public Access were ever accidentally disabled, the policy would immediately re-expose the data. Defense in depth means removing both the override and the underlying misconfiguration.

Before vs. After Comparison

Before Remediation	After Remediation
Block All Public Access: OFF	Block All Public Access: ON
Bucket policy: Principal: "*", s3:GetObject	Bucket policy: Deleted
employee-data.txt → publicly readable	employee-data.txt → AccessDenied
AWS Config: NONCOMPLIANT	AWS Config: COMPLIANT
Incognito access: File content visible	Incognito access: XML AccessDenied error
Any internet user can read all objects	Only authenticated, authorized IAM identities can access objects

9. Key Learnings

- Block Public Access and bucket policies are independent controls — and you need both. Block Public Access is the safety override, but the bucket policy is the underlying configuration. Disabling the override without also cleaning up the permissive policy leaves a ticking time bomb. Always remediate both layers.

- AWS Config is genuinely useful for compliance detection, but it needs to be set up proactively. Just like CloudTrail in Lab 1, Config only detects misconfigurations you've told it to watch for. In a real environment, a baseline set of Config rules covering S3, IAM, CloudTrail, and Security Groups should be part of the standard account setup.

- The acknowledgment checkbox during bucket creation is meaningful — but insufficient. AWS makes you actively acknowledge that you're making a potentially public bucket. That friction is good. But it's not enough: organizations should enforce Block Public Access at the AWS Organizations level using SCPs so that individual account owners can't disable it even if they want to.

- What surprised me most: the incognito browser test was the most viscerally effective part of this lab. Seeing real file content — a salary figure, a name — load instantly in a browser with no login is a very different experience from reading that public buckets are

a risk. The visual confirmation makes the vulnerability concrete in a way that a compliance flag alone doesn't.

- Remediation validation closes the loop. Running the Config re-evaluation after the fix and confirming COMPLIANT status, then testing the incognito access again, gave me confidence that the fix actually worked. In a real incident response scenario, this validation is what you document and report to stakeholders — not just that you made a change, but that you verified its effect.

10. Conclusion

This lab covered the full lifecycle of an S3 data exposure incident: from misconfiguration to live public exposure, automated detection via compliance rules, controlled remediation, and verified validation. The scenario — a developer creates a bucket, disables public access protection thinking it's needed, uploads data, and walks away — is not a hypothetical. It's a pattern that's played out in real organizations dozens of times.

What this exercise reinforced for me is that cloud security isn't just about knowing which buttons to press. It's about understanding why each control exists and what happens when layers are removed. Block Public Access exists because bucket policies can be misconfigured. Config rules exist because manual auditing doesn't scale. Incognito testing exists because you need to verify from the attacker's perspective, not just the admin's.

For anyone working toward a cloud security or SOC role, S3 exposure findings are some of the most common real-world alerts you'll encounter. Knowing how to assess the severity, trace the misconfiguration back to its root cause (policy vs. Block Public Access), remediate both layers, and validate the fix is a directly applicable skill set. This lab builds exactly that muscle memory.

> *Key takeaway: In cloud environments, data exposure doesn't require an attacker to exploit a vulnerability — sometimes you accidentally make the door public and leave it open. The best defenses are preventive controls (Block Public Access at the org level), detective controls (Config rules, Macie), and a validation habit that tests every fix from the outside in.*